Document Number:	P0054R00
Date:	2015-09-12
Project:	Programming Language C++, Evolution
Revises:	none
Reply to:	gorn@microsoft.com

P0054R00: Coroutines: Reports from the field

Introduction

An experimental version of the compiler supporting coroutines (aka resumable functions N4134, N4286, N4402) was out in the wild for nearly a year now. This paper proposes changes based on the feedback received from customers experimenting with it, the feedback from WG21 committee members, and learning from the experience of converting large shipping application to use coroutines for asynchronous operations. The updated wording is provided in a separate paper P0057R00.

Defects

In revision 4 of the resumable function proposal (N4402), the requirements on the return type of `initial_suspend`, `final_suspend` and `yield_value` member functions were changed from having to return an `Awaitable` type, to a type contextually convertible to bool.

```
Before N4402:

struct coro {
  struct promise_type {
    std::experimental::suspend_never initial_suspend() { return {}; }
    std::experimental::suspend_always  final_suspend() { return {}; }
    ...
  }
};

After N4402:

struct coro {
  struct promise_type {
    bool initial_suspend() { return false; }
    bool final_suspend() { return false; }
    ...
  }
};
```

While this made simple code slightly simpler, it also made it **impossible** to write the correct code for less trivial coroutine scenarios. Consider the following:

Immediately Scheduled Coroutines

Imagine a case where an invocation of a coroutine immediately schedules it to execute on a thread pool and yields control back to the caller. Murphy's law guarantees that a scheduler will execute the coroutine to completion and deallocate all the memory associated with the coroutine state even prior to `initial_suspend` call returning.

Prior to N4402, initial suspend point was defined in terms of operator `await`, i.e. `await $promise.initial_suspend()`. await operator mechanics took care of the race when the resumption of the coroutine happens before `await_suspend` completes, by preparing the coroutine for the resumption prior to the invocation of the `await_suspend`. A check via `await_ready` was used to avoid this potentially expensive preparation if the result of the computation was already available. (Expensive here means a few extra store operations, such as saving non-volatile registers in use, and storing an address or an index of the resume point, for example).

To handle the race, N4402's initial_suspend() would need to add the logic similar to that of the await. Hence we propose to go back to defining initial suspend point via `await $promise.initial_suspend()`.

Final suspend racing with coroutine_handle::destroy()

Imagine a case where a library developer would like to combine the allocation of a `future` shared state N4527/[futures.state] with the allocation of the coroutine state in the case when the coroutine returns the future.

```
future<int> deep_thought() {
  await 7'500'000'000h;
  return 42;
}
```

This will require an atomic reference count in the shared state / promise. One reference will be held by the future, to make sure it can examine the shared state even when the coroutine has completed execution, and another reference will be from the coroutine itself, since it does not want its memory deallocated by the future destructor while in the middle of the execution.

Future destructor will decrement the reference count in the shared state and if the reference goes to zero will invoke `destroy()` member of the `coroutine_handle` to free the state. Similarly, when the coroutine reaches the final suspend point, it decrements the reference and if it happens to be zero, meaning the future is gone and no longer requires the shared state, the coroutine should not suspend at the final point and proceeds straight to the end and destroy its state.

However, it is possible that the future's destructor has decremented the reference count immediately after final_suspend() checked that the reference count is not zero, but before final_suspend returned. This is very similar to the race described in he previous section and the solution is the same: we need to rely on await operator to resolve it. Here is how the correct `final_suspend` would look like.

```
struct promise_type : shared_state<T> { // refcount is in the shared state
  auto final_suspend() {
    struct awaiter {
       promise_type * me;
       bool await_ready() { return false; }
       void await_resume() {}
       bool await_suspend(coroutine_handle<>) {
          auto need_suspending = (me->decrement_refcount() > 0);
          return need_suspending;
       }
    };
    return awaiter{this};
  }
  ...
};
```

Asynchronous generator yield_value races with consumer going away

Consider an asynchronous generator:

```
async_generator<int> quick_thinker() {
  for (;;) {
    await 1ms;
    yield 42;
  }
}
```

We need to coordinate between a producer i.e. the coroutine shown above and a consumer that is whomever is holding on to an async_generator object. In this particular case a consumer owns the producer. After a consumer decides to go away, meaning async_generator destructor runs, a well behaved producer should stop its activity and release the resources it uses, moreover the producer should not attempt to resume the consumer as it is gone. Thus the producer in its `value needs` needs to make a determination: if the consumer is alive, give it the value and resume it, otherwise, the producer coroutine need to cancel itself by invoking coroutine_handle::destroy() on itself. This could be implemented correctly with pre N4402 version. Again the fix is to revert to pre-N4402 behavior and define `yield expr` in terms of `await $promise.yield_value` as `await_suspend` allows to concurrent resumption of the coroutine either via `resume()` and `destroy()`. With the fix, implemenation of `yield_value` would look like:

```
template <typename T>
struct async_generator {
  struct promise_type {
    T const * yielded_value;
    coroutine_handle<> consumer;
    ...
    auto yield_value(T const& v) {
      struct awaiter {
         promise_type * me;
         bool await_ready() { return false; }
         T const & await_resume() { return *me->yielded_value; }
         void await_suspend(coroutine_handle<> myself) {
           ... if consumer is gone => myself.destroy();
           ... otherwise           => consumer.resume();
         }
```

```
      };
      yielded_value = &v;
      return awaiter{this};
    }
  };
  ...
};
```

operator await

Currently `await expression` uses a range-based for like lookup for three member or non-member functions called `await_suspend`, `await_ready` and `await_resume`. This has not been always the case. An earlier iteration of the resumable functions proposal that never got to be an N-numbered paper had defined `operator await` that had to return an awaitable object that has member functions `await_suspend`, `await_ready` and `await_resume`.

Let's compare how await adapters used to look like and how they look in N4134 and beyond.

```
auto sleep_for(chrono::system_clock::duration d) {
  struct result_t {
    chrono::system_clock::duration d;

    auto operator await() {
       struct awaiter {
          chrono::system_clock::duration duration;
             ...
          awaiter(chrono::system_clock::duration d) : duration(d){}
          bool await_ready() const { return duration.count() <= 0; }
          void await_resume() {}
          void await_suspend(std::experimental::coroutine_handle<> resume_cb){...}
       };
       return awaiter{d};
     }
  };
  return result_t{d};
}
```

The authors felt that this was too much boilerplate code and one more local class than desired, hence the N4134 offered a range-based-for like lookup instead of operator await. Indeed under N4134 rules the code is simpler.

```
auto sleep_for(chrono::system_clock::duration d) {
  struct awaiter {
     chrono::system_clock::duration duration;
       ...
     awaiter(chrono::system_clock::duration d) : duration(d){}
     bool await_ready() const { return duration.count() <= 0; }
     void await_resume() {}
     void await_suspend(std::experimental::coroutine_handle<> resume_cb){...}
  };
  return awaiter{d};
}
```

However this simplification removed one powerful ability that was enabled by `operator await`. It was no longer possible for library author to rely on a temporary object on a coroutine frame during the await expansion that can persist for the duration of await expression and can be used to carry state between `await_ready`, `await_suspend` and `await_resume` functions. The only form that remained that allowed this was when awaitable was returned from a function, such as in the example of `sleep_for` above.

Consider this straightforward, but incorrect awaitable adapter for `boost::future`.

```
template <typename T> bool await_ready(boost::future<T>& f) { return f.ready(); }
template <typename T> T await_resume(boost::future<T>& f) { return f.get(); }
template <typename T> void await_suspend(boost::future<T>& f, coroutine_handle<> cb) {
  f.then([cb](auto&&){ cb(); });
}
```

The problem is that as of version 1.59, future.then returns a future that blocks in the destructor. Thus coroutine, after subscribing to the completion of f.then will block at the last curly brace of await_suspend waiting for the destructor that will block until the future is ready prevening coroutine from suspending. Though in the case of boost, we can fix boost .then, in case of other libraries it may not be possible to change them to adapt await within time constraints. Having `operator await` would have addressed this problem.

To make sure that a future returned from the `.then` won't block the suspend, we need to extend its life for the duration of await expression. With operator

`await` we can do it easily:

```
template <typename T>
auto operator await(boost::future<T> & f) {
  struct awaiter {
    future<T>* me;
    future<T> keep_this;
    bool await_ready() { return me->ready(); }
    T await_resume() { return me->get(); }
    void await_suspend(coroutine_handle<> cb) {
       keep_this = f.then([cb](auto&&){ cb(); });
    }
  }
  return awaiter{this, {}};
}
```

Another case for `operator await` is adapter efficiency. Imagine that we want to do a lean future that allows multiple coroutines to subscribe their awaits on lean_future's `.then` and make the subscription operation via `.then` to be `noexcept` and not perform any memory allocations. `operator await` makes this possible:

```
template <typename T>
auto operator await(lean_future<T> & f) {
  struct awaiter {
    lean_future<T>* me;
    lean_future<T>::intrusive_link link;
    bool await_ready() { return me->ready(); }
    T await_resume() { return me->get(); }
    void await_suspend(coroutine_handle<> cb) {
       keep_this = f.then(&me->link);
    }
  }
  return awaiter{this, {}};
}
```

Since `operator await` enables library to control the temporary that lives for the duration of the await-expression, library writer can include in the temporary the intrusive_list::link so that it can be directly linked into the intrusive list in the lean_future. That removes an allocation and a failure mode. In kernel mode of operating system, in game development those are important properties.

Now, Some of these tecniques are possible today with N4134, but only with awaitables that are temporaries returned from a function, like in sleep_for in earlier in this section. Having `operator await` fixes existing assymmetry that different awaitables have different expressive power.

We would like to bring back operator `await` with an improvement that will result in less boilerplate code. The proposed change is to make an `operator await` to be implicitly defined for a class that has `await_suspend`, `await_resume` and `await_ready` and it is defined as an identity function. It returns the object itself. With this approach, we can now address the problems described above and retain the concise style available today.

As a bonus, it is now possible to write an await adapter for chrono::duration that we have been sneakily using throughout this paper that allows us to write `await 10ms`. Behold:

```
template <class Rep, class Period>
auto operator await(chrono::duration<Rep, Period> d) {
  struct awaiter {
     chrono::system_clock::duration duration;
       ...
     awaiter(chrono::system_clock::duration d) : duration(d){}
     bool await_ready() const { return duration.count() <= 0; }
     void await_resume() {}
     void await_suspend(std::experimental::coroutine_handle<> resume_cb){...}
  };
  return awaiter{d};
}
```

It's 2015, why are we still making statements that can't serve as expressions...?

No good reason. Pre-N4402 it was an expression. N4402 did a lot of "simplifications" that are now being undone. Making yield a statement as opposed to an expression was one of the "simplifications".

The suggested change here is to let `yield expr` and `yield {expr}` be expressions not statements with the same precedence as a `throw expr`.

```
  assignment-expression:
```

```
    conditional-expression
    logical-or-expression assignment-operator initializer-clause
    throw-expression
    yield-expression
```

This precedence would allow `yield` to be used with `comma operator` and at the same time to be able to write `yield 1 + 2` without surprising parsing `(yield 1) + 2`.

Side effect of this change and making `yield_value` return awaitable as required to fix the defect described in previous section opens the possibility for library writers to invent and implement semantics for *yield-expresion* returning something back into the coroutine enabling two way communication between the generator and the consumer.

Allocators be gone

Authors have received a strongly worded feedback that it is highly undesirable to make a language feature dependent on std::allocators. Other language features rely on allocating via `operator new` and use overloading of `operator new` as a way to customize allocations for classes that require specialized allocation strategies.

To address this concern and bring the coroutines more in line with other language features, if a coroutine requires dynamic memory allocation for its state, it will call `operator new` and customization of allocations could be done by overloading operator new. We implemented this change and discovered that most of the user code that customized coroutine allocations with stateless allocators shrunk significantly.

Before:

```
template <typename T, typename... Ts>
struct coroutine_traits<generator<T>, use_counting_allocator_t, Ts...> {
   template <typename T>
   struct counting_allocator {
      std::allocator<T> inner;
      using value_type = T;

      T* allocate(std::size_t n) {
        bytes_allocated += n * sizeof(T);
        return inner.allocate(n);
      }
      void deallocate(T* p, std::size_t n) {
        bytes_freed += n * sizeof(T);
        inner.deallocate(p, n);
      }
    };

    template <typename... Us>
    static auto get_allocator(Us&&...) {
        return counting_allocator<char>{};
    }
    using promise_type = typename generator<T>::promise_type;
};
```

After

```
template <typename T, typename... Ts>
struct coroutine_traits<generator<T>, use_counting_allocator_t, Ts...> {
    struct promise_type : generator<T>::promise_type {
        void* operator new(size_t size) {
            bytes_allocated += size * sizeof(T);
            return ::operator new(size);
        };
        void operator delete(void* p, size_t size) {
            bytes_freed += size * sizeof(T);
            ::operator delete(p, size);
        }
    };
};
```

Note that in the get_allocator example, get_allocator it is getting all of the coroutine arguments so that if it is a stateful allocator it can pick up required information from the arguments. The suggested change preserves an ability to pass information to an allocation routine, but, it keeps the simple case (non-stateful) simple by using the following rule: if the coroutine promise defines an `operator new` that take just `size_t`, it will be used to allocate the memory for the coroutine, otherwise, the compiler will use the `new-expression` of the form `promise_type::operator new(required-size, all of the arguments passed to a coroutine)`. The latter forms allows for an overloaded new to extract

required allocator parameters.

Finally, to preserve parity with N4402 with respect to allocators, we need to address coroutine operations in the environment where allocation functions cannot throw. N4402 was determining the need for special handling of allocations by checking if `get_return_object_on_allocation_failure` static member function was present in `coroutine_traits`, we suggest to move it to `coroutine_promise` and use `std::nothrow_t&` form of `operator new` in this case.

Before:

```
struct coro {
  struct promise_type {
    coro get_return_object();
    ...
  };
};

template <typename... Args> struct coroutine_traits<coro, Args...> {
  static coro get_return_object_on_allocation_failure();
  using promise_type = coro::promise_type;
};
```

After:

```
struct coro {
  struct promise_type {
    static coro get_return_object_on_allocation_failure();
    coro get_return_object();
    ...
  };
};
```

With this changes, not only we remove dependency of coroutines on std::allocator and friends, we also moved most of the functionality present in `coroutine_traits` that deal with allocation concerns into the coroutine promise making specializing `coroutine_traits` unnecessary in majority of cases. The only remaining case for using `coroutine_traits` is when one defines a coroutine promise for a type that belong to some pre-existing library that cannot be altered.

On the radar

This section describe some changes we are exploring at the moment, but, did not have time to implement and experiment with. We plan to proposem at the next meeting. They are listed here for an opportunity for early feedback.

promise_type::await_transform

One of the pattern in use with frameworks using `.then` is to use a cancellation flag / token to be passed to a function and furnished to every `.then` to facilitate cancellation.

When porting the code to use `await`, every await expression was wrapped with an awaitable adapter that would take an existing awaitable and augment it to check the cancellation flag and cancel the coroutine if required.

```
  auto bytesRead = await conn.Read(buf, len);

     would become

  auto bytesRead = await CheckCancel(cancelToken, conn.Read(buf, len));
```

Adding CheckCancel at every await site is cumbersome and error prone.

We would like to provide an ability for the coroutine type author to specify an `await_transform` member in the promise_type of the coroutine. If present, every `await expr` in that coroutine would be as if it was `await $promise.await_transform(expr)`.

Besides helping with cancellation, await_transform has other uses:

debugging / tracing / performance measuring

With an appropriate await_transform, coroutine can trace/log when it is suspended, when it is resumed, whether suspension was avoided due to await_ready being true, etc. This allows debugging tools accumulate information for asynchronous activity visualization. It can be used for capturing the traces for problem or performance analysis.

undo yet another "simplification" from N4402

In N4402 whether `await` is allowed or not in the coroutine is tied to whether the coroutine promise defines `return_value/return_void` with argumentation that coroutines that `await` on something have an eventual value return value, but, generators do not. This restriction was introduces in N4402 to help detect mistakes at compile time when `await` is used in coroutines that don't support it.

await_transform allows library author trivially to specify a compile check whether coroutine is allowed or not to use await and limitation introduced in N4402 is no longer required.

Exploring design space

Resumable expressions paper (N4453) has a compelling example of *magically* transforming a function template into a coroutine depending on the `OutputIterator` supplied.

```
template <class OutputIterator> void fib(int n, OutputIterator out) {
  int a = 0;
  int b = 1;
  while (n-- > 0) {
    *out++ = a;
    auto next = a + b;
    a = b;
    b = next;
  }
}
```

Automatically Awaited Awaitables

This section sketches out an idea how coroutines can evolve to support the scenario above. The idea is simple. If a function returns an object of type that is marked with `auto await`, an `await` is injected into the calling function. For the example above, dereferencing of an iterator would return a proxy that has an overloaded operator `=` that returns *automatically awaited awaitable*.

```
auto MyProxy::operator=(int output) {
  struct Awaitable auto await { ... };
  return Awaitable{...};
}
```

Thus an expression `*out++ = a` will become `await (*out++ = a)`. Awaitable will transfer supplied value `a` to the consumer and suspend the function `fib` until the next value is requested. **Note that this has not been designed, implemented and there is no immediate plan to pursue this approach.**

One concern with this approach is that it interferes with composability of awaitable expressions. If `f()` and `g()` returns awaitables, we would like to be able to transform awaitable in questions prior to applying `await` to them. For example, evaluation of `await f() + await g()` reduces concurrency as it would be more beneficial to execute it as `await (f() + g())`, where the result of `+` is a composite awaitable that will wait until both results of `f()` and `g()` are ready and will resume the coroutine providing the sum of the eventual results of f() and g().

Another concern is that it is now near impossible for the reader to figure out whether function is a coroutine or not unless we can audit every function call, implicit conversion, overloaded operator in the body of the function and figuire out if it can return *automatically awaited awaitable*.

Moreover, even though coroutines allow asynchrous code to be written nearly as simple as synchronous, they do not eliminate the need to think about and properly design the lifetime of the asynchronous activity. Const-ref parameters `const&` that are perfectly fine to consume in a normal function may result in a crash, information disclosure and more if the function is a coroutine which lifetime extends beyond the lifetime of the object bound to that `const&` parameter.

Acknowledgements

Kavya Kotacherry, Daveed Vandevoorde, Richard Smith, Jens Maurer, Lewis Baker, Kirk, Shoop, Hartmut Kaiser, Kenny Kerr, Artur Laksberg, Jim Radigan, Chandler Carruth, Gabriel Dos Reis, Deon Brewis, Jonathan Caves, James McNellis, Stephan T. Lavavej, Herb Sutter, Pablo Halpern, Robert Schumacher, Viktor Tong, Michael Wong, Niklas Gustafsson, Nick Maliwacki, Vladimir Petter, Shahms King, Slava Kuznetsov, Tongari J, Lawrence Crowl, Valentin Isac and many more who contributed.

References

P0055r00: On Interactions Between Coroutines and Networking (http://wg21.link/P0055R00)
P0057r00: Wording for Coroutines, Revision 3 (http://wg21.link/P0057R00)

N4527: Working Draft, Standard for Programming Language C++ (http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4527.pdf)
N4402: Resumable Functions (revision 4) (https://isocpp.org/files/papers/N4402.pdf)
N4286: Resumable Functions (revision 3) (http://open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4286.pdf)
N4134: Resumable Functions v2 (http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4134.pdf)
N4453: Resumable Expressions (http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4453.pdf)